E-MASTERS PROGRAMME IN COMPUTER SCIENCE: AI & ML · IIIT HYDERABAD

CodeGen

Hybrid RAG Retrieval + LoRA Fine-Tuning on a Small Code LLM

Team Members(Group 2)

N Anjan Karthik Chandra
Jakka Rithusravya
Darshna Kumawat

Mentor

Mr. Sajid Ansari

§

ABSTRACT

Problem Statement

Natural-language-to-query interfaces are well studied for relational SQL — the Spider benchmark alone underlies hundreds of published systems — but equivalent tooling for MongoDB's document-oriented Query Language (MQL) is nascent: dedicated datasets and frameworks (TEND, Spider, EvoMQL) appeared only in 2025–2026. This project converts Spider into 1,114 natural-language-to-MQL pairs spanning 19 databases and builds a compact pipeline around Qwen2.5-Coder-0.5B-Instruct, combining LoRA fine-tuning (r=16, α=32, full attention + MLP) with a hybrid retriever that fuses dense semantic search (FAISS + BAAI/bge-small-en-v1.5) with structural, MQL-aware feature matching. Evaluated with BLEU, ROUGE, CodeBERTScore, and a token-overlap Response Accuracy metric on a 100-sample held-out set, LoRA fine-tuning alone raises Response Accuracy from 33% to 76% and improves every metric measured, at a cost of roughly 39% additional GPU latency. Retrieval augmentation, evaluated separately across five top-k settings, reduces every metric relative to the fine-tuned baseline — a negative result reported in full rather than omitted. The work contributes a reproducible Spider-to-MQL pipeline, a hybrid semantic/structural retriever tailored to MQL syntax, and an empirical account of where fine-tuning and retrieval each help — and where they don't — for small, open code language models.

1,114
Q → MQL pairs

33% → 76%
Response Accuracy (LoRA)

RAG: net negative
on every metric tested

+39%
GPU latency from LoRA

ROADMAP

What We'll Cover

01 Related Work

02 Dataset: Spider → MongoDB

03 System Architecture

04 Methodology

05 Hybrid Retrieval Design

06 LoRA Fine-Tuning Setup

07 Evaluation Framework

08 Results & Findings

09 Auxiliary Capabilities

10 Acknowledgement

R

LITERATURE

Related Work

Area	Prior Work	Relevance Here
Text-to-SQL foundations	Spider (Yu et al., EMNLP 2018)	Source benchmark this project's MQL dataset is derived from
Retrieval-augmented generation	RAG(Lewis et al., NeurIPS 2020)	Retrieves few-shot MQL exemplars instead of text passages
Parameter-efficient fine-tuning	LoRA (Hu et al., ICLR 2022)	Technique used to specialize Qwen2.5-Coder
Code-aware evaluation	CodeBERT (Feng et al., EMNLP-Findings 2020)	Backbone for the CodeBERTScore metric used here
Text-to-NoSQL (2025)	TEND / SMART ; MultiTEND	Closest prior systems: fine-tuned small LM + RAG for NL→NoSQL
MongoDB-specific benchmark	Spider (Özer et al., 2025)	First Spider-style cross-domain dataset for MongoDB
NL → MQL generation (2026)	EvoMQL (Ye et al.)	Closest prior NL→MQL method — 3B model, self-evolving refinement

i

Positioning: unlike TEND/SMART and EvoMQL, this project derives its benchmark directly from Spider, uses a sub-1B open model, and separately ablates hybrid retrieval against LoRA fine-tuning to isolate which component actually helps.

DATA

Spider, Converted to MongoDB

1,114
question / MQL pairs

19
source databases

3
fields per record

```
{ question, database_id,
generated_mongodb_query }
```
Loaded via DatasetLoader → flattened for LoRA training & retrieval indexing

SAMPLE RECORDS (database_id: concert_singer)

Q Show name, country, age for all singers, oldest to youngest.

```
→ db.singer.find({}, {Name:1,Country:1,Age:1,_id:0}).sort({Age:-1})
```

Q What is the average, minimum, and maximum age of singers from France?

```
→
db.singer.aggregate([{$match:{Country:"France"}},{$group:{_id:null,avg_age:{$
avg:"$Age"},...}}])
```

TOP DATABASES BY SAMPLE COUNT

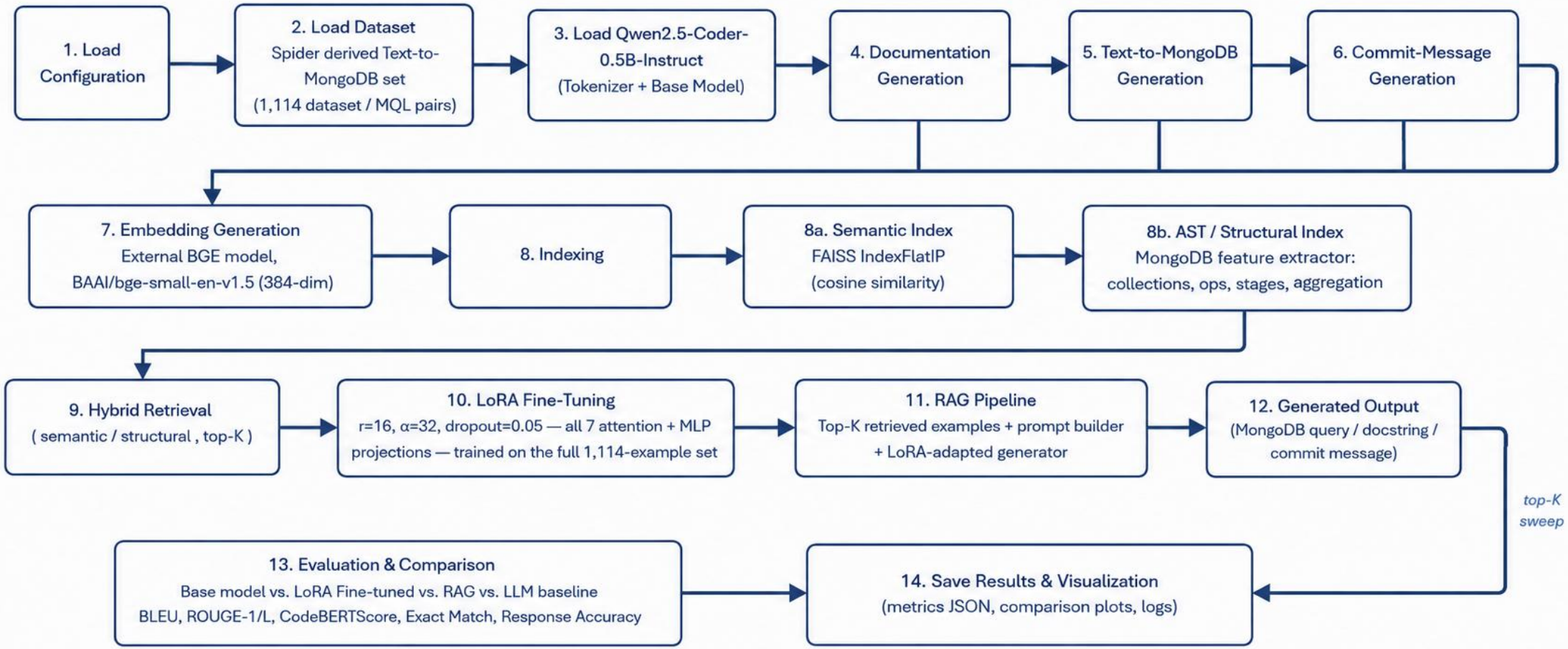
Database	Samples
world_1	120
wta_1	114
car_1	92
cre_Doc_Template_Mgt	84
dog_kennels	82
flight_2	80

ARCHITECTURE

System & Technology Stack

Layer	Technology
Base generator model	Qwen2.5-Coder-0.5B-Instruct
Fine-tuning	PEFT LoRA— r=16, α=32, on attention + MLP projections
Embedding model	BAAI/bge-small-en-v1.5 (dense, L2-normalized)
Dense retrieval	FAISS IndexFlatIP— exact inner-product / cosine search
Structural retrieval	Custom AST-style feature index (collection + operator features)
Fusion	Min-max normalized score fusion — semantic / structural
Evaluation backbone	microsoft/codebert-base (local checkpoint) for CodeBERTScore
Metrics	BLEU · ROUGE-1/L · CodeBERTScore · Response Accuracy · Latency
Orchestration	4 notebooks: main · baseline_model · finetuned_model · spider_evaluation

End-to-End Pipeline (CodeGen Project)



RETRIEVAL

Hybrid Retrieval Design

S Semantic path (dense)

Encode query with BAAI/bge-small-en-v1.5, L2-normalized.

Search FAISS IndexFlatIP (exact inner product = cosine) for the top 2×k nearest question/query pairs.

Captures intent — “how many…”, “average of…” — even across different wordings.

A Structural path (AST-style)

Regex-based feature extraction over MQL: collection:<name>, op:<find|aggregate|match|group…>.

Features indexed in an inverted index for fast structural lookup.

Captures query shape — an aggregate pipeline retrieves other aggregate pipelines.

```
score(doc) = 0.3 × norm(semantic_score) + 0.7 × norm(structural_score)
min-max normalized independently per query, top-k re-ranked from the merged pool
```

i Why hybrid? Dense embeddings match meaning; structural features match MQL shape — combining both retrieves examples aligned on intent and syntax.

FINE-TUNING

LoRA Setup

Hyperparameter	Value
Rank (r)	16
Alpha (α)	32
Dropout	0.05
Target modules	q/k/v/o_proj, gate/up/down_proj (full attn + MLP)
Trainable fraction	100% of LoRA parameters
Batch size	1
Learning rate	2e-4
Epochs	1

CAUSAL LM LOSS MASKING

prompt tokens — label = −100 (ignored)

completion tokens (MQL) — loss computed

```
### Task: Generate MongoDB Query (MQL)
### Question:
{question}
### MongoDB Query:
db.collection.find(…)  ← loss here
```

A single-epoch LoRA pass adapts a 0.5B-parameter code model to MQL syntax across all 1,114 examples — loss is computed only on the query completion.

EVALUATION

How We Measure Quality

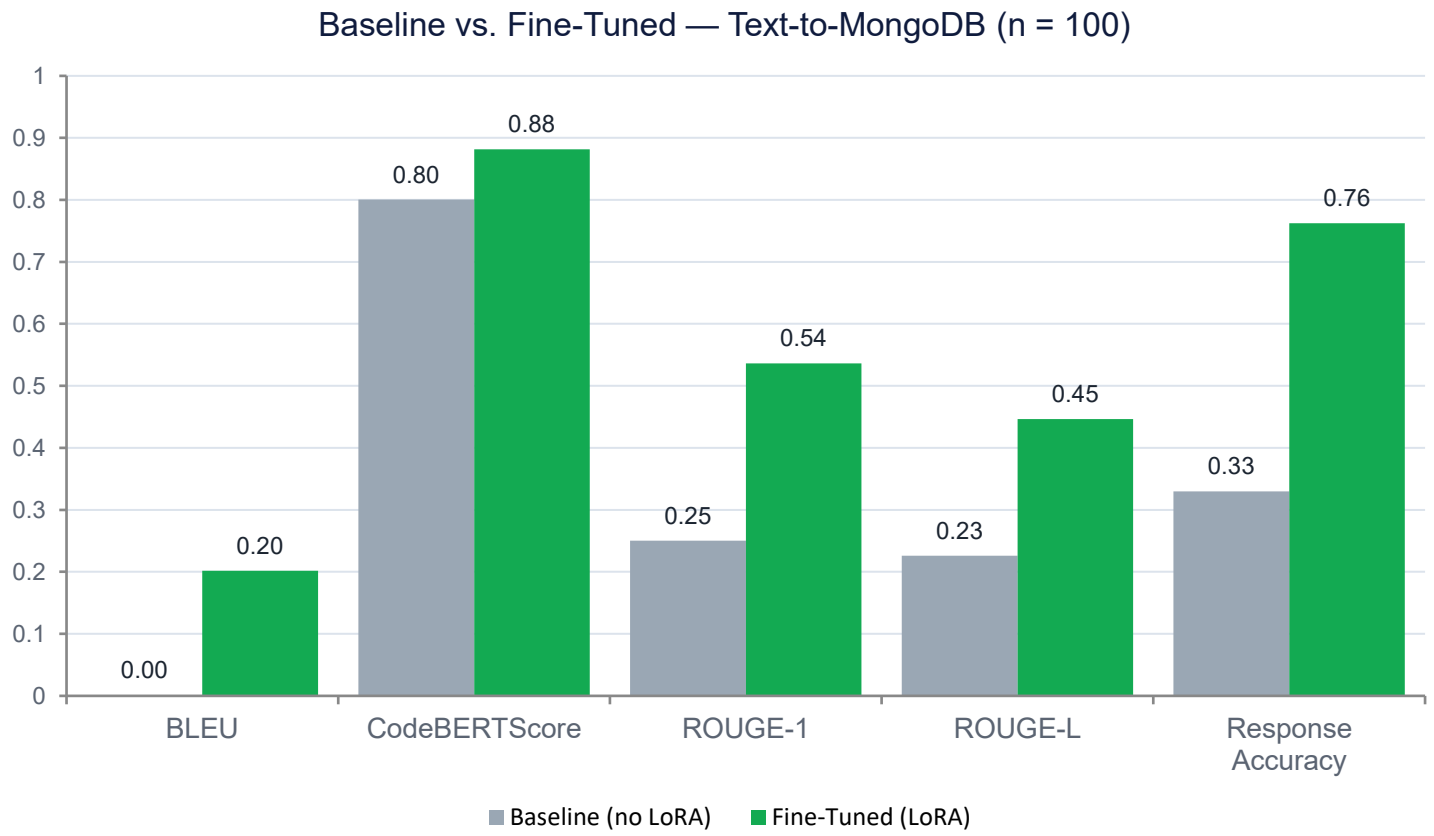
Metric	What it measures	Notes
BLEU	n-gram precision vs. gold MQL (smoothed, corpus-level)	Tokenized on MQL punctuation
ROUGE-1 / ROUGE-L	Unigram / longest-common-subsequence overlap	F-measure, stemmed
CodeBERTScore	Semantic similarity via local CodeBERT embeddings (BERTScore F1)	Required fixing a silent num_layers bug that zeroed every score
Response Accuracy	Token-overlap recall: |gold ∩ pred| / |gold|	Lenient — not execution-based
Latency	Wall-clock ms per generated query	Measured end-to-end, GPU

Generated Metrics : Baseline & Fine-tuned (LoRa)

Model	BLEU	CodeBERTScore	ROUGE-1	ROUGE-L	Response_Accuracy	Avg_Latency_ms
Baseline	0.0008	0.8004	0.2504	0.2263	0.3297	38890.28
Fine-tuned (LoRA)	0.2019	0.8817	0.5360	0.4462	0.7624	54060.22

RESULTS · 100-SAMPLE BENCHMARK

Fine-Tuning Impact



+0.201

BLEU (0.001 → 0.202)

+0.433

Response Accuracy (33% → 76%)

n=100

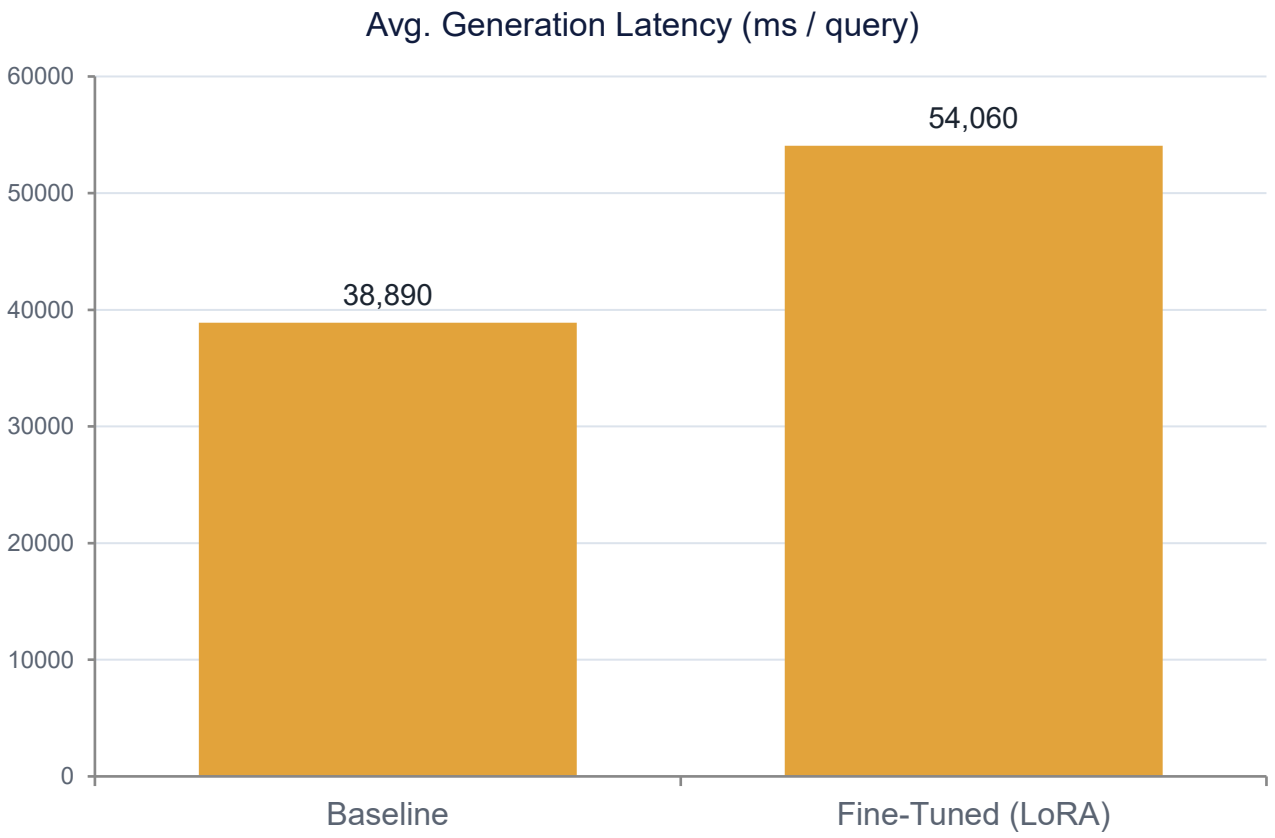
held-out questions

EVAL SET SIZE

Baseline	100
Fine-tuned (LoRA)	100

RESULTS · 100-SAMPLE BENCHMARK

Latency & Efficiency Trade-off

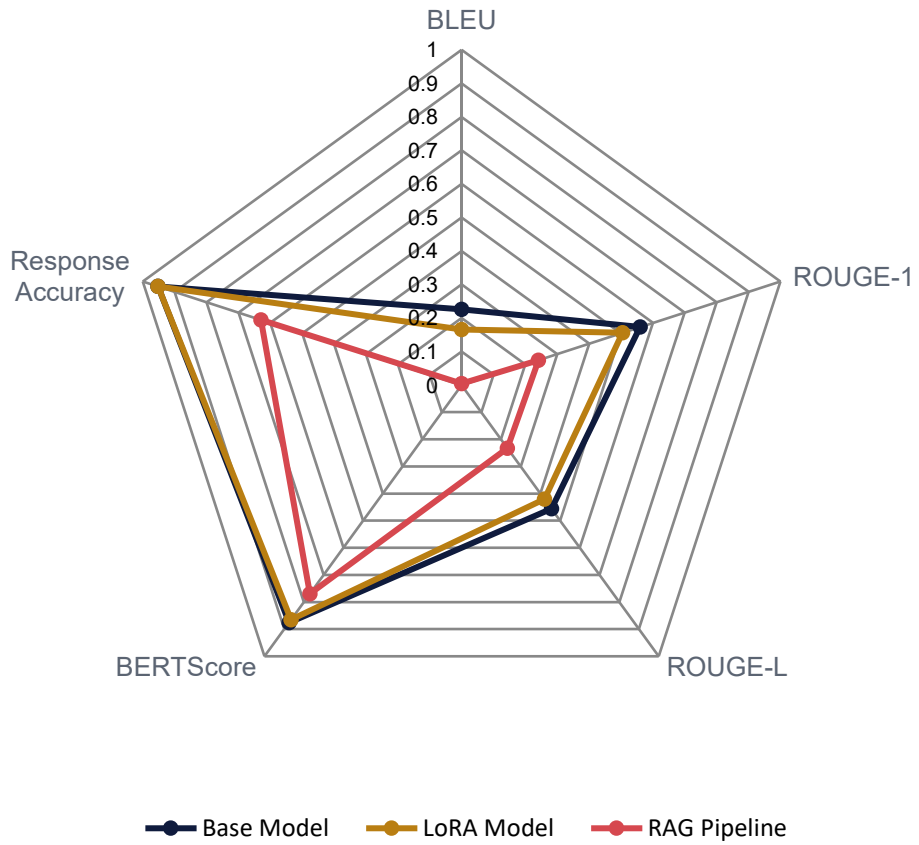


+39%

inference time added by LoRA adapters

RESULTS · 5-SAMPLE QUICK EVAL

Base vs. LoRA vs. RAG Pipeline



Model	BLEU	ROUGE-1	BERTScore	Resp. Acc.	Latency
Base Model	0.226	0.560	0.876	0.952	20.4s
LoRA Model	0.165	0.505	0.865	0.952	25.4s
RAG Pipeline	**0.004**	**0.241**	**0.769**	**0.629**	**33.9s**

! On this evaluation, adding retrieval hurts every single metric — RAG_Pipeline scores lowest on BLEU, ROUGE, BERTScore, and Response Accuracy, and is also the slowest of the three (33.9s vs. 20–25s).

RAG PIPELINE — 5-SAMPLE RUNS

Comparison	Examples
RAG Pipeline vs. Baseline (Base Model)	5
RAG Pipeline vs. Fine-tuned (LoRA)	5
Fine-tuned (LoRA), without RAG	5

FINDING

Gain Analysis — Base, Fine-Tuned & RAG

Vs Base Model

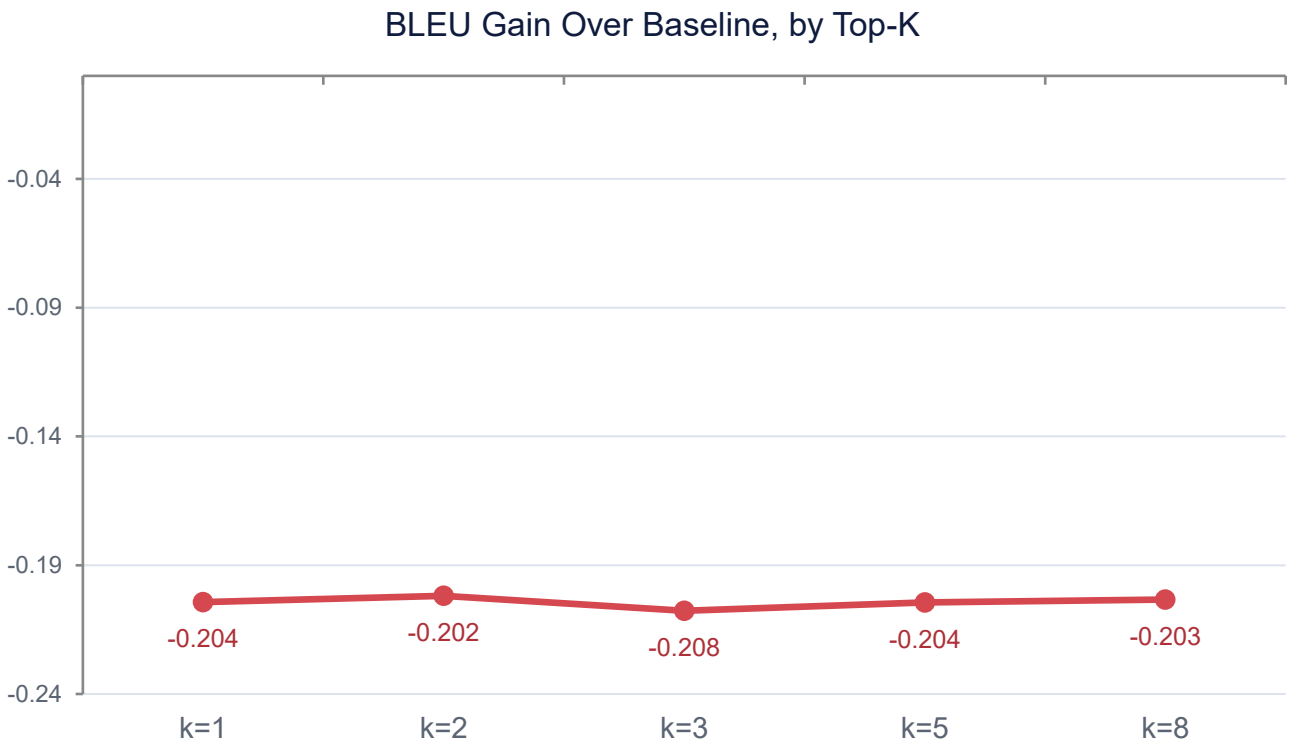
Metric	LoRA Gain	RAG Gain	RAG vs. LoRA
BLEU	−0.0605	−0.2215	−0.1610
ROUGE-1	−0.0550	−0.3185	−0.2635
ROUGE-L	−0.0361	−0.2231	−0.1870
BERTScore	−0.0109	−0.1065	−0.0956
Response_Accuracy	+0.0000	−0.3227	−0.3227

Vs Fine Tuned(LoRa Model)

Metric	Base Gain	RAG Gain
BLEU	+0.0605	−0.1610
ROUGE-1	+0.0550	−0.2635
ROUGE-L	+0.0361	−0.1870
BERTScore	+0.0109	−0.0956
Response_Accuracy	+0.0000	−0.3227

FINDING

RAG Top-K Sweep



Top-K	BLEU	ROUGE-1	BLEU Δ	ROUGE-1 Δ
1	0.0057	0.2694	−0.204	−0.230
2 (best)	**0.0081**	**0.2468**	**−0.202**	**−0.253**
3	0.0023	0.1864	−0.208	−0.313
5	0.0056	0.1802	−0.204	−0.319
8	0.0067	0.2089	−0.203	−0.291

Best top_k by BLEU gain: k = 2 — but "best" still means "least harmful." No k tested turns the gain positive.

! Likely cause: retrieved few-shot examples add prompt-format drift for a model already specialized by LoRA on this exact distribution — retrieval competes with fine-tuning rather than complementing it. Measured on only 5 questions, so treat magnitudes as directional.

RESULTS · QUALITATIVE

What Each Model Actually Generates

Question	Reference (gold)	Base Model	LoRA (fine-tuned)	RAG Pipeline
How many singers do we have?	db.singer.countDocuments()	db.actors.countDocuments()db.singer.aggregate([{$group:{_id:"count"}}, {$project…	db.actors.countDocuments()db.singer.aggregate([{$group:{_id:"count"}}, {$project…	db.actors.aggregate([{$group: {_id: "$Singer_ID"}}, {$count: "count"}])SELECT co…
What is the total number of singers?	db.singer.countDocuments()	db.actors.countDocuments()db.singer.aggregate([{$group:{_id:"count"}}, {$sum:_id…	db.actors.countDocuments()db.singer.aggregate([{$group: {_id: null, count: {$sum…	db.actors.aggregate([{$group: {_id: null}}, {$count: "sum"}]) # $group to aggreg…
Show name, country, age for all singers ordered by age from the oldest to the yo…	db.singer.find({}, {Name:1, Country:1, Age:1, _id:0}).sort({Age:-1})	db.actors.find({}, {Name:1, Country:0, Age:-1}).sort({Age:-2})db.singer.aggr…	db.actors.find({}, {Name:1, Country:2, Age:3}).sort({Age:-1})db.singer.aggre…	db.averageround.aggregate([{"$addFields": {"sum_age": {$sum: "$age"}}},{"$divide…

BEYOND QUERY GENERATION

Auxiliary Capabilities

D Documentation Generation

5-shot prompted DocGenerator turns a code snippet into a natural-language docstring.

```
def calculate_area(radius):
    return 3.14159 * radius ** 2

→ "Calculates the area of a circle
   given its radius."
```

C Commit Message Generation

CommitMessageGenerator writes conventional commit messages (feat / fix / docs…) directly from a git diff.

```
diff --git a/auth.py b/auth.py
+ def refresh_token(user):
+     …

→ "feat: add token refresh
   handling for auth flow"
```

i The same GenerationPipeline is reused across tasks — only the prompt template changes — showing the fine-tuned adapter's utility isn't limited to MQL generation.

WITH THANKS

Acknowledgments

Sajid Ansari	Project Mentor — for technical mentorship on the RAG and fine-tuning design.
IIIT Hyderabad & NSE TalentSprint	For the E-Masters program structure, curriculum, and computing resources.
Open-source & dataset providers	Spider (Yu et al.), Qwen Team, HuggingFace PEFT/Transformers, FAISS, and BAAI (BGE embeddings) — the tooling this project builds on.
Group 2 teammates	N Anjan Karthik Chandra, Jakka Rithusravya and Darshna Kumawat — for collaboration across data prep, modeling, and evaluation.

Thank You